Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3064 Sistemas Operativos
Anggelie Lizeth Velásquez Asencio 221181

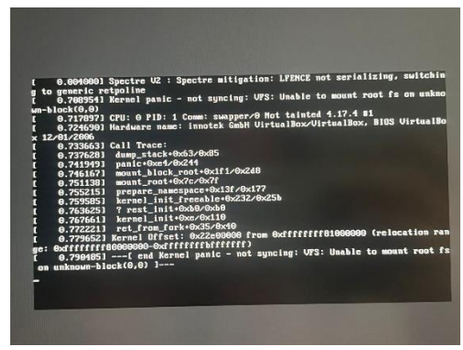
Ciclo 1 de 2026

Laboratorio 02

Ejercicio 1 (10 puntos)

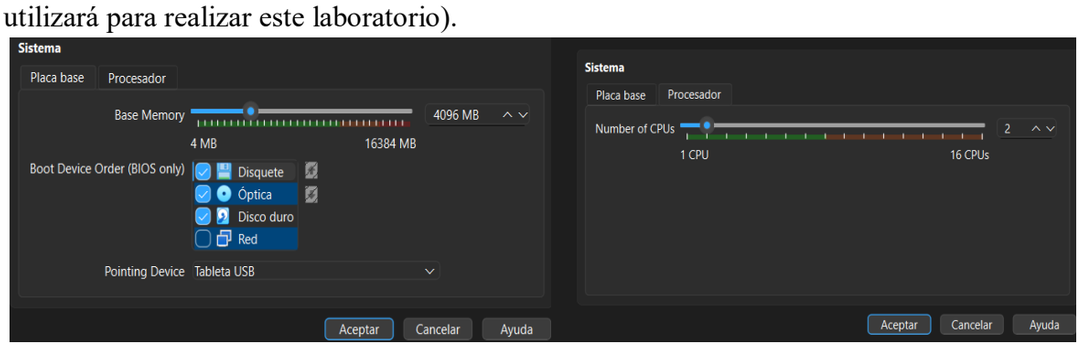
a. Inicie con su máquina virtual apagada, y revise que esté usando más de un procesador (recomendable, cuatro) en el menú de configuración de VirtualBox (o el Software de virtualización utilizado).
De no ser así, configúrela para que use más de un procesador (o cámbiese de computadora *host* a una multinúcleo).

```
This message is shown once a day. To disable it please create the
/root/.hushlogin file.
root@Anggelie:~# lscpu | egrep 'CPU\(s\)|Core|Thread|Model name'
nproc
CPU(s):                                  16
On-line CPU(s) list:                     0-15
Model name:                              AMD Ryzen 9 6900HX with Radeon Graphics
Thread(s) per core:                      2
Core(s) per socket:                      8
16
root@Anggelie:~#
```



Se verifico que el equipo host es multinúcleo (16 CPUs lógicos) con lscpu/nproc. La VM presentó Kernel Panic recurrente, por lo que los experimentos se ejecutaron en Ubuntu nativo manteniendo evidencia de multiprocesamiento.

b. Adjunte una captura de pantalla de los parámetros de configuración de su máquina virtual (la que utilizará para realizar este laboratorio).



c. Cree un programa en C que ejecute cuatro fork()s consecutivos. Luego cree otro programa en C que ejecute fork() dentro de un ciclo for de cuatro iteraciones.

d. Responda las siguientes preguntas

- ¿Cuántos procesos se crean en cada uno de los programas?
  - Ambos crean $2^4 = 16$ procesos en total
  - Procesos nuevos creados: 15
- ¿Por qué hay tantos procesos en ambos programas cuando uno tiene cuatro llamadas fork() y el otro sólo tiene una?

- En el for no es un solo fork, es un fork por interación(4 forks). Además, cada fork() duplica el proceso actual, así que el crecimiento es exponencial: después de 1 fork hay 2, después 2 forks hay 4, después 3 hay 8, después 4 hay 16.

Ejercicio 2 (20 puntos)

a. Cree un programa en C que #incluya los encabezados <stdio.h> y <time.h>. Este programa deberá ejecutar tres ciclos for consecutivos, de un millón de iteraciones cada uno. Ninguno de los ciclos deberá desplegar ni hacer nada.

b. Declare, al principio de su programa, dos variables de tipo clock_t. Ejecute la función clock() justo antes del primer ciclo for, almacenando el resultado en una variable de tipo clock_t. También ejecute la llamada a clock() justo después del último for y almacene el resultado en la segunda variable clock_t.

c. En el programa, luego de los tres ciclos, almacene en una variable de tipo double el resultado de la resta entre las variables clock_t (la variable que está antes se resta a la que está después). Tome en cuenta que, por almacenarse en una variable double, la resta debe ser *casteada* a este tipo.

d. Haga que el programa despliegue el contenido de la variable double en pantalla. El especificador de formato que debe usar es %f, que sirve para números de punto flotante.

e. Ejecute su programa varias veces (tres o cinco veces suele exhibir el comportamiento deseado) y apunte los resultados de cada vez.

f. Cree un nuevo programa en C que #incluya los encabezados <stdio.h>, <time.h>, <unistd.h> y <sys/wait.h>. Al principio del programa declare tres variables de tipo pid_t y dos variables de tipo clock_t.

g. Este programa hará lo mismo que en el primero, pero de forma concurrente (recuerde que esto no es sinónimo de "paralelo"). Para lograrlo comience por realizar y almacenar el resultado de una llamada a clock() justo antes de un fork(), y almacene el resultado de cada llamada en las variables con tipos correspondientes.

h. Haga que el proceso hijo realice otro fork(), y que este nuevo proceso (sería el proceso nieto) haga también un nuevo fork().

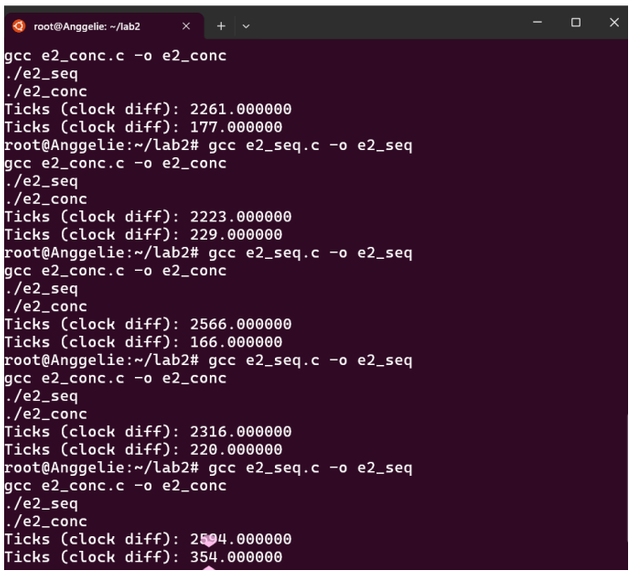
i. El proceso bisnieto (el creado por el fork() más anidado) debe realizar un ciclo for de un millón de iteraciones que no hagan nada. El proceso nieto debe realizar lo mismo que el bisnieto en el inciso anterior, pero de forma exclusiva. Es decir, en el else del if que restringe el for al inicio de este inciso al proceso bisnieto. Asegúrese de que el proceso nieto espere, luego de completar su for, a que termine el proceso bisnieto con wait(NULL).

j. El proceso hijo debe realizar lo mismo, y en las mismas condiciones, que el proceso nieto en el inciso anterior.

k. En el proceso raíz o padre exclusivamente (es decir, en el else menos anidado) espere a que termine la ejecución del proceso hijo y luego ejecute clock(). Despliegue la diferencia entre las variables clock_t y deje terminar el programa.

l. Ejecute este programa la misma cantidad de veces que ejecutó el programa no concurrente.

Secuencial (ticks)	2261	2223	2566	2316	2594
Concurrente con forks (ticks)	177	229	166	220	354



En estas ejecuciones, el secuancial reportó más ticks que el concurrente; sin embargo, clock() mide tiempo de CPU del proceso y el uso de múltiples procesos pueden hacer que el valor del padre no refleje todo el trabajo total del sistema.

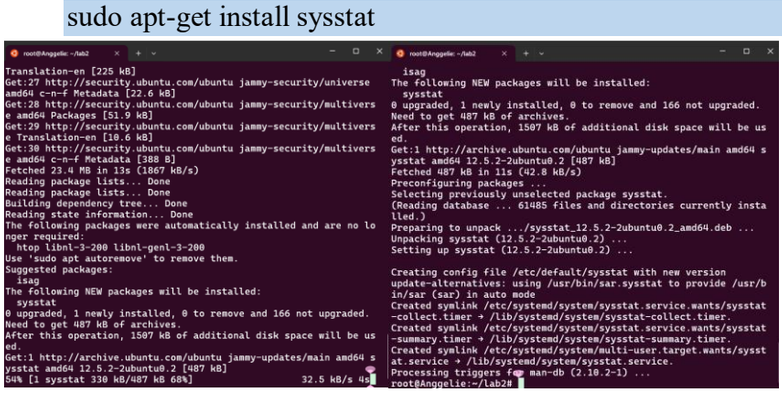
m. Compare los resultados de tiempos de cada uno de sus programas y responda:

- ¿Cuál, en general, toma tiempos más largos?
  - Normalmente el ocurrente con forks suele reportar más (porque crea procesos y hay overhead) o valores raros por como clock() mide CPU.
- ¿Qué causa la diferencia de tiempo, o por qué se tarda más el que se tarda más?
  - Overhead de fork() + wait() + planificación del SO. Además, la concurrencia no es igual a paralelo si hay 1 core o el scheduler reparte, intercalan.

Ejercicio 3 (20 puntos)

a. Descargue e instale el paquete sysstat usando apt-get en una terminal (o el manejador de paquetes del sistema operativo que tenga, *e.g.*, yum o dpkg):

sudo apt-get install sysstat



b. Investigue un poco sobre los **cambios de contexto voluntarios e involuntarios**. Agregue a su documento una breve explicación de su investigación.

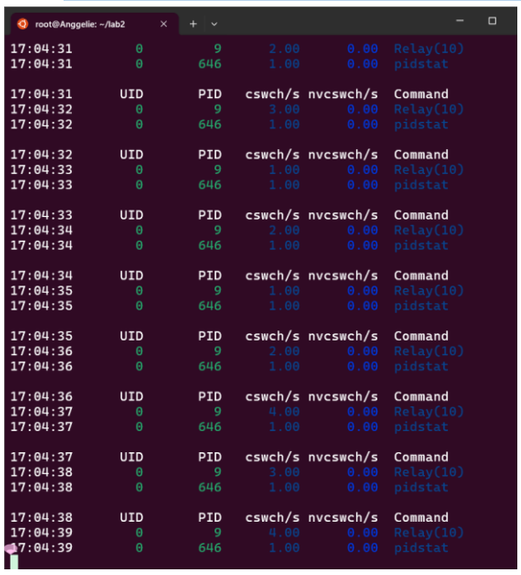
- Cambio de contexto voluntario: el proceso cede CPU
- Cambio de contexto involuntario: el scheduler lo interrumpe

c. Abra una segunda terminal y diríjase en ella al directorio donde estén los programas que hizo en el ejercicio anterior.

d. Coloque las terminales en pantalla de forma que pueda ver ambas a la vez.

e. Ejecute el comando pidstat en la primera terminal con la opción -w para desplegar el número de cambios de contexto que se realizan por proceso; y agregue el parámetro 1 al final para que se realice este reporte cada segundo. La instrucción debe verse así:

pidstat –w 1



f. Realice acciones en la interfaz gráfica (*e.g.*, mueva la ventana donde ejecutó pidstat o abra una ventana nueva) y observe el efecto que esto tiene sobre el proceso Xorg.
En la terminal que está ejecutando pidstat, teclee cosas y observe qué procesos aparecen o responden ante estas interacciones.

• ¿Qué tipo de cambios de contexto incrementa notablemente en cada caso, y por qué?

- GUI/teclado: suelen aumentar voluntarios porque hay más I/O y procesos esperando eventos.
- Carga de CPU: aumentan más los involuntarios porque el scheduler preempte por quantum/competencia.

g. Modifique los programas de su ejercicio anterior para que desplieguen el índice de sus ciclos con cada iteración. En la segunda terminal, ejecute cada programa y tome el tiempo en segundos que toma cada uno en terminar.

h. Cancele la ejecución de pidstat con Ctrl-C. Escriba (pero no ejecute) en la primera terminal el siguiente comando:

pidstat –w X 1

donde X es la cantidad (o el entero superior inmediato) de **segundos** que calculó para el programa sin
fork().

i. Escriba (pero no ejecute) en la segunda terminal el comando para ejecución del programa sin fork()s.

j. Ejecute primero el pidstat y luego, lo más inmediatamente posible, el programa sin fork()s, y espere a que este último termine.

k. Anote el número de cambios de contexto de cada tipo para el proceso correspondiente a la ejecución de su programa. Si no percibe resultados, reste 1 a X en pidstat e intente de nuevo.

l. Repita los pasos anteriores para el programa con fork()s que realizó en el ejercicio anterior.

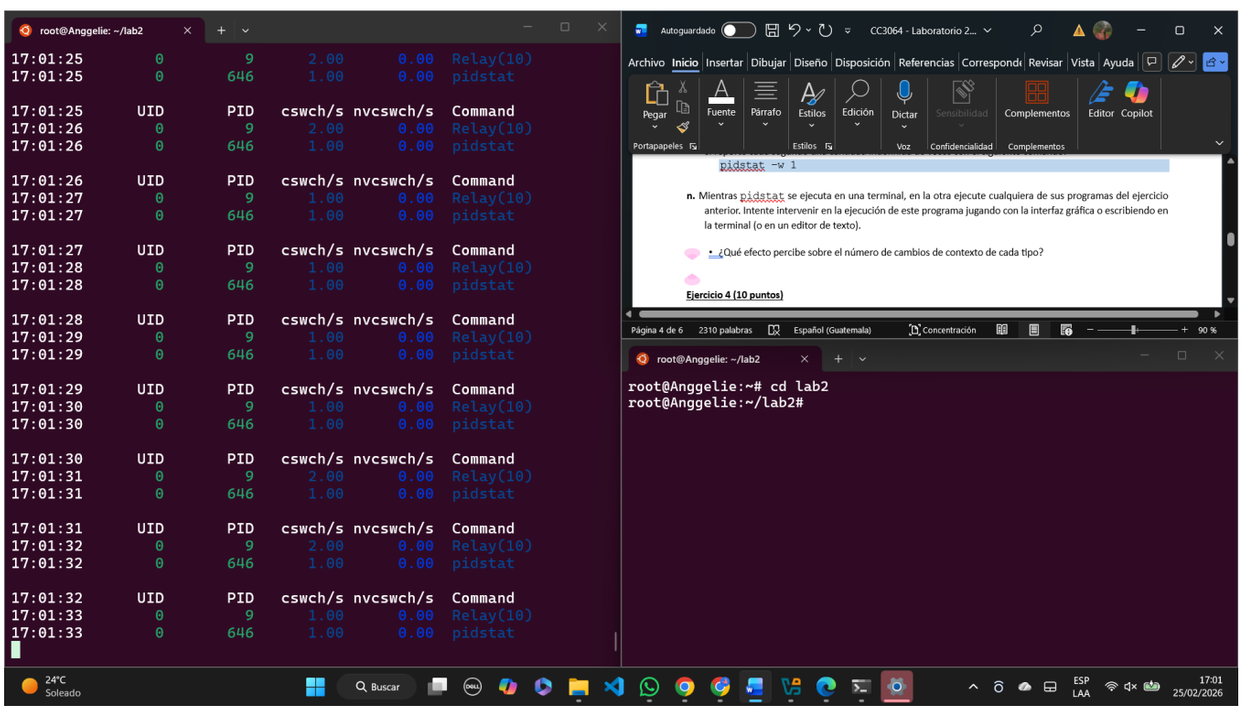
- ¿Qué diferencia hay en el número y tipo de cambios de contexto de entre programas? Muestra varios procesos, y cada uno tendrá su propio patrón de cambios de contexto.
- ¿A qué puede atribuir los cambios de contexto voluntarios realizados por sus programas? Por wait(), por bloqueos a escribir en terminal, por I/O.
- ¿A qué puede atribuir los cambios de contexto involuntarios realizados por sus programas? Por competencia de CPU, quantum, preempción.
- ¿Por qué el reporte de cambios de contexto para su programa con fork()s muestra cuatro procesos, uno de los cuales reporta cero cambios de contexto? Porque uno puede terminar rapidísimo antes de que pidstat lo capture o porque casi no ejecutó.

m. Vuelva a realizar los procedimientos de medición de cambios de contexto, pero esta vez haga que se muestre el reporte cada segundo una cantidad indefinida de veces con el siguiente comando:

```
pidstat –w 1
```

n. Mientras pidstat se ejecuta en una terminal, en la otra ejecute cualquiera de sus programas del ejercicio anterior. Intente intervenir en la ejecución de este programa jugando con la interfaz gráfica o escribiendo en la terminal (o en un editor de texto).

¿Qué efecto percibe sobre el número de cambios de contexto de cada tipo? Al ejecutar pidstat mientras corre el programa, se observa que los cambios de contexto varían según la carga del sistema y la interacción del usuario. Cuando el programa solo realiza cálculos (loops sin I/O), predominan los cambios de contexto involuntarios, ya que el scheduler interrumpe al proceso cuando se le acaba el tiempo de CPU o cuando otro proceso necesita ejecutarse.En cambio, cuando se interactúa con la interfaz gráfica o se escribe en la terminal, aumentan los cambios de contexto voluntarios, debido a que los procesos realizan operaciones de entrada/salida y se bloquean esperando eventos, cediendo voluntariamente la CPU. En resumen, la interacción del usuario incrementa principalmente los cambios voluntarios, mientras que la competencia por CPU incrementa los involuntarios.



Ejercicio 4 (10 puntos)

a. Escriba un programa en C que realice un fork(). En el proceso hijo debe desplegarse un mensaje en pantalla únicamente, y en el proceso padre (exclusivamente) debe ejecutarse un ciclo while infinito.

b. Abra dos terminales.

c. Ejecute el programa escrito en el inciso anterior en una de las terminales. En la otra ejecute el siguiente comando:

```
ps –ael
```

En el resultado verá dos procesos con el nombre de su programa, uno de los cuales tendrá añadido <defunct>,si no logra verlo a la primera vez de ejecutar el comando, pruebe ejecutarlo varias veces. Este proceso también desplegará una Z en la segunda columna.

- ¿Qué significa la Z y a qué se debe? La Z indica un proceso zombie (<defunct>). El hijo ya terminó, pero el padre sigue ejecutándose y no ha llamado a wait(), por lo que el sistema conserva la entrada del hijo en la tabla de procesos hasta que el padre recoja su estado.

d. Ahora modifique su programa para que en lugar de desplegar un mensaje en el proceso hijo despliegue el conteo de 1 a 4,000,000. El objetivo es que los despliegues en pantalla tomen entre 5 y 15 segundos, por lo que puede incrementar el límite del conteo si es necesario.

e. Ejecute su programa en una de las dos terminales y en la otra vuelva a ejecutar ps –ael. Anote los números de proceso de tanto el padre como el hijo.

f. Repita el inciso anterior de modo que éste y el próximo paso se realicen antes de que termine el conteo. En la terminal donde ejecutó el comando ps ejecute el siguiente comando:

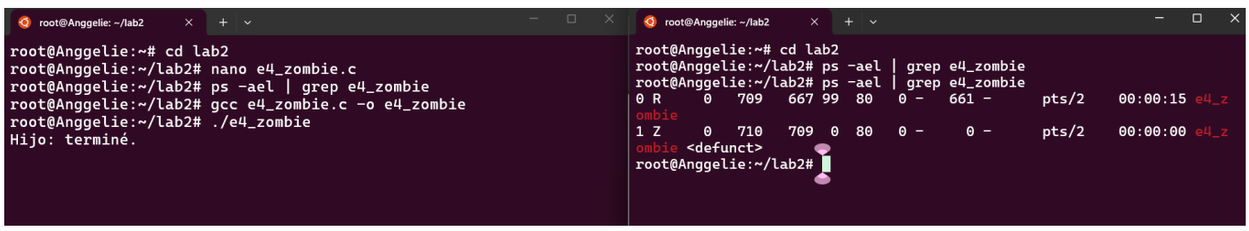
kill -9 <numproc>

donde <numproc> debe ser reemplazado por el número de proceso padre.

- ¿Qué sucede en la ventana donde ejecutó su programa? Al ejecutar kill -9 <PID_padre>, el proceso padre termina inmediatamente. El hijo continúa ejecutándose y queda huérfano, por lo que el sistema lo reasigna automáticamente a otro padre.

g. Vuelva a ejecutar ps –ael.

- ¿Quién es el padre del proceso que quedó huérfano? El padre pasa a ser PID 1 (normalmente systemd en Ubuntu). Se observa porque el PPID del hijo cambia a 1 en ps -ael.



Ejercicio 5 (40 puntos)

Escriba un programa en C llamado ipc.c que reciba desde terminal un número que llamaremos n y una letra que llamaremos x (investigue sobre int argc, char** argv). **Dos instancias de este programa se ejecutarán concurrentemente**. Para mejor entendimiento de lo que se les solicita investiguen acerca del uso de pipes (anónimos y nombrados) y memoria compartida en c. Recordemos que parte de nuestro aprendizaje es también que el estudiante investigue acerca de lo que se le solicita. Lea las preguntas al final de este ejercicio, le servirán también como guía de lo que se espera que su programa pueda realizar

El código de su programa deberá abrir un espacio de memoria compartida y luego enviar el *file descriptor* de este espacio a la otra instancia de sí mismo. Si el espacio de memoria compartida ya fue abierto por la otra instancia deberá detectarlo, recibir el *file descriptor* de su otra instancia y luego desplegarlo en pantalla. NO debe usar este *file descriptor* para *mappear* la memoria compartida.

El programa, sin importar que haya creado el espacio de memoria compartida o que se haya adherido a un espacio existente, eventualmente *mappeará* la memoria compartida a un puntero. Luego del *mappeo*, deberá engendrar un proceso usando fork().

El proceso padre iniciará un ciclo for con un número de iteraciones igual al tamaño asignado a la memoria compartida. En el ciclo se revisará si el número de iteración es divisible entre n, en cuyo caso se enviará x al proceso hijo, "notificándole" que pasó por una iteración divisible entre n. El proceso hijo deberá estar pendiente de la comunicación del proceso padre y deberá escribir cada x recibida en la memoria compartida.

Una vez concluido el ciclo for, el proceso padre esperará al hijo mientras éste escribe, luego desplegará el contenido de la memoria compartida y, por último, cerrará las comunicaciones. **Nota**: se calificará programación defensiva contra posibles errores en el manejo de fork()s, *pipes*, etc., y que sus programas tomen las medidas adecuadas en el manejo de sus comunicaciones y archivos (cerrar archivos abiertos, quitar el *mappeo* de memoria compartida, etc.).
Finalmente, escriba un segundo programa que únicamente realice un fork(), y que en cada una de sus ramas ejecute, con execl, el programa ipc con diferentes símbolos para x. El resultado debe ser que se despliegue el contenido de memoria compartida haciendo evidente que ambas instancias de ipc han escrito en ella.

A continuación, un ejemplo de cómo podría verse el *output* de su último programa, que ejecuta dos instancias de ipc:

```
os@debian:~$ ./ipcRunner.o
I am a
I am b
b: Created new shared mem obj 4
b: Ptr created with value 0xb7879000
b: Initialized shared mem obj
a: Shared mem obj already created
a: Received shm fd: -1
a: Ptr created with value 0xb78b9000

b: Shared memory has: bbbbbbbbbbbbbbbbbbbbbbbbbb
os@debian:~$
a: Shared memory has: bbbbbbbbbbbbbbbbbbbbbbbbbbaaaaaaaaaaaaaaaaaaaaaaaaaa
```

Responda las siguientes preguntas.

- ¿Qué diferencia hay entre realizar comunicación usando memoria compartida en lugar de usando un archivo de texto común y corriente?
  - Memoria compartida: comunicación rápida (RAM), sin I/O de disco ya que permite el intercambio directo entre procesos.
  - Archivo: hay I/O, locking más complicado, latencia y puedes tener inconsistencias si ambos escriben.
- ¿Por qué no se debe usar el *file descriptor* de la memoria compartida producido por otra instancia para realizar el mmap?
  - Porque el objetico es que cada instancia abra su propia referencia y controle permisos/estado.
- ¿Es posible enviar el *output* de un programa ejecutado con exec a otro proceso por medio de un *pipe*?

Investigue y explique cómo funciona este mecanismo en la terminal (*e.g.,* la ejecución de ls | less).

  - Si, porque la Shell crea un pipe, luego hace fork(), redirige stdout del primer proceso al pipe (dup2) y redirige stdin del segundo proceso desde el pipe. Después ejecuta ambos con exec.
- ¿Cómo puede asegurarse de que ya se ha abierto un espacio de memoria compartida con un nombre determinado? Investigue y explique ***errno***.
  - Usas shm_open(name, O_CREAT | O_EXCL, ...). Si falla con -1 y errno == EEXIST, entonces ya existía.
  - errno es una variable global que indica la razón del error del último syscall.
- ¿Qué pasa si se ejecuta shm_unlink cuando hay procesos que todavía están usando la memoria compartida?
  - Se elimina el nombre, pero la memoria sigue viva mientras haya procesos con el objeto abierto/mapeado. Se libera definitivamente cuando el último proceso cierre y des mapee.
- ¿Cómo puede referirse al contenido de un espacio en memoria al que apunta un puntero? Observe que su programa deberá tener alguna forma de saber hasta dónde ha escrito su otra instancia en la memoria compartida para no escribir sobre ello.
  - Con un struct en la shm y un índice compartido idx, cada escritura hace data[idx]=… y luego incrementa idx.
- Imagine que una ejecución de su programa sufre un error que termina la ejecución prematuramente, dejando el espacio de memoria compartida abierto y provocando que nuevas ejecuciones se queden esperando el *file descriptor* del espacio de memoria compartida. ¿Cómo puede liberar el espacio de memoria compartida "manualmente"?

  - Desde la terminal se pueden ver los objetos usando ls/dev/shm y borrarlos usando rm/dev/shm o también mejor volver a correr un programa que llame shm_unlik(SHM_NAME)

- Observe que el programa que ejecute dos instancias de ipc.c debe cuidar que una instancia no termine mucho antes que la otra para evitar que ambas instancias abran y cierren su propio espacio de memoria compartida. ¿Aproximadamente cuánto tiempo toma la realización de un fork()? Investigue y aplique usleep.
  - Depende del sistema, pero suele ser del orden de microsegundos a pocos milisegundos.
  - Cuando usleep(50000) duerme duarante 50ms y ayuda a evitar condiciones de carrera como que ambas instancias intenten crear/cerrar shm demasiado rápido.

```
root@Anggelie: ~/lab2

root@Anggelie:~/lab2# kill -9 <PID_PADRE>
-bash: syntax error near unexpected token `newline'
root@Anggelie:~/lab2# nano ipc.c
root@Anggelie:~/lab2# nano ipcRunner.c
root@Anggelie:~/lab2# gcc ipc.c -o ipc
gcc ipcRunner.c -o ipcRunner
./ipcRunner
a: Created new shared mem obj
b: Shared mem obj already created
b: Received shm fd from other instance: 5
b: Shared memory has: aaaaaaaaaaaababababababababababababaababababab
abababababababababababababababababababababababababababababababababab
abababababababababababababababababaababababababababababababababababa
bababababababababababababababababbababababaababababbababababababab
abababababababababababababababababaababababababababababababababababa
bababababababababababababababababababababababababababababababababababa
bababababababababababababababababababababababababababababababababababa
bababababababababababababababababababababababababababababababababababa
bababababababababababababababababbbbbbbbbbbbbbbbbbbbababababab
abababababababababababababababababababaababababababababababababababa
bababababababababababababababaababababababababababababababababababab
abababababababababababababababababababababababababababababababaaba
aa
a: Shared memory has: aaaaaaaaaaaababababababababababababaababababab
abababababababababababababababababababababababababababababababababab
abababababababababababababababababaababababababababababababababababa
bababababababababababababababababbababababaababababbababababababab
abababababababababababababababababaababababababababababababababababa
bababababababababababababababababababababababababababababababababababa
bababababababababababababababababababababababababababababababababababa
bababababababababababababababababababababababababababababababababababa
bababababababababababababababababbbbbbbbbbbbbbbbbbbbababababab
abababababababababababababababababababaababababababababababababababa
bababababababababababababababaababababababababababababababababababab
abababababababababababababababababababababababababababababababaaba
aa
```

  - Se imprime el file descriptor recibido únicamente como evidencia del IPC; el mmap() se realiza con el descriptor abierto por la propia instancia, como solicita el enunciado.